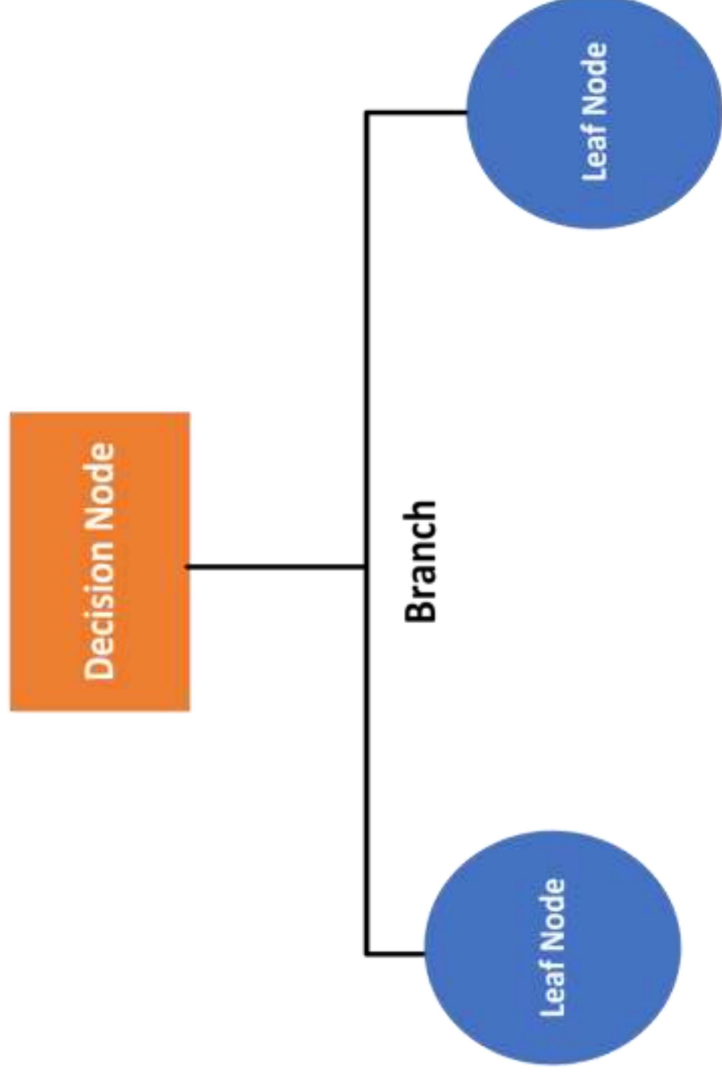


Decision Tree



Structure Decision Tree

- Just like a tree has: **Trunk** → Main decision (Root Node)
- **Branches** → Possible choices (Yes/No paths)
- **Leaves** → Final decisions or outcomes (Leaf Nodes)
- Example: Question: Is the salary greater than \$80,000?
(Root Node)
 - If yes → Next question: Is the commute less than 1 hour?
 - If no → Check paid time off (More than 4 weeks?)

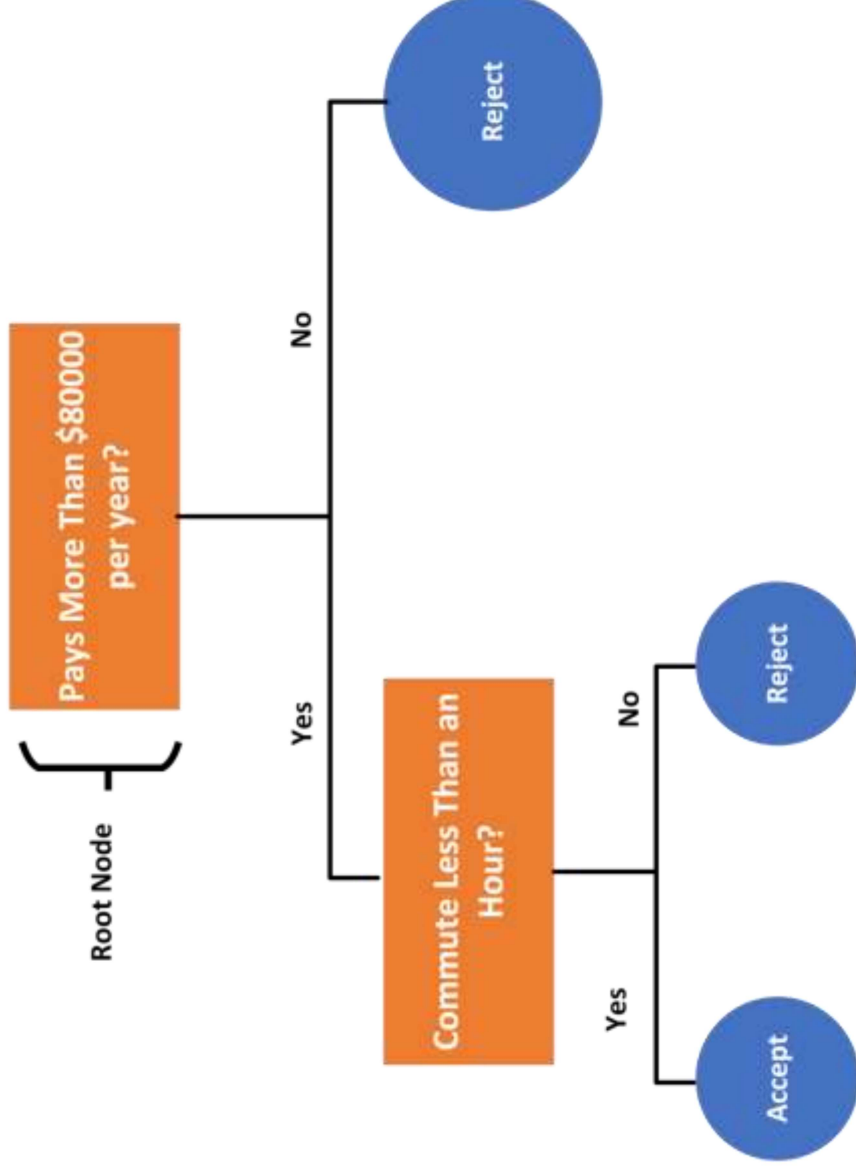
Decision Nodes & Leaf Nodes

- **Decision Nodes** = Where you make a choice (e.g., salary, commute time)
- **Leaf Nodes** = Final outcomes (e.g., accept or reject a job offer)
- The **flow** from decision to outcome can be thought of as **if-then-else rules**.

Real Life Example of Decision Tree

- Deciding to take a new job: Is the salary $> \$80,000$?
 - Yes: Check commute time.
 - No: Check paid time off.
- Is the commute < 1 hour?
 - Yes: Accept job.
 - No: Reject job.

Real Life Example of Decision Tree



Decision Tree Uses

- **Classification Problems:** If the output is a category (e.g., yes/no, colors)
- **Regression Problems:** If the output is a number (e.g., age, income)
- Easy to explain the logic to non-technical people.
- Commonly used in **legal and compliance settings** for transparency.

Classification Tree

- Decision tree where the depended variable is categorical

Customer	Independent Variable			Default		
	Amount	Grade	Purpose			
	James S.	\$10000	C		Business	Yes
	Carol J.	\$40000	B		Home	Yes
	Sarah P.	\$15000	A		Car	No
John J.	\$20000	B	Home	No	Dependent Variable	

Regression Tree

- Decision tree where the depended variable is continous

Independent Variable				Dependent Variable
Customer	Profession	Length	Degree	
James S.	Programmer	5	Master's	
Carol J.	Engineer	15	Bachelor's	
Sarah P.	Professor	10	Doctorate	
John J.	Programmer	17	Bachelor's	

How is Classification Tree Built?

- **Classification trees** use a process called **recursive partitioning**.
- The goal is to split the data into smaller groups to make items within each group as **similar** as possible.
- Each split helps us build the tree by creating **decision nodes** and **leaf nodes**.

Example of Bank Loan Decision Tree

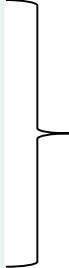
- Imagine we work at a bank with data on 30 loans:
Variables: **Income**, **Loan Amount**, and whether they **defaulted** on the loan (yes/no).
- The goal is to predict if future loans will result in default.

Bank Data Loan

Independent Variable

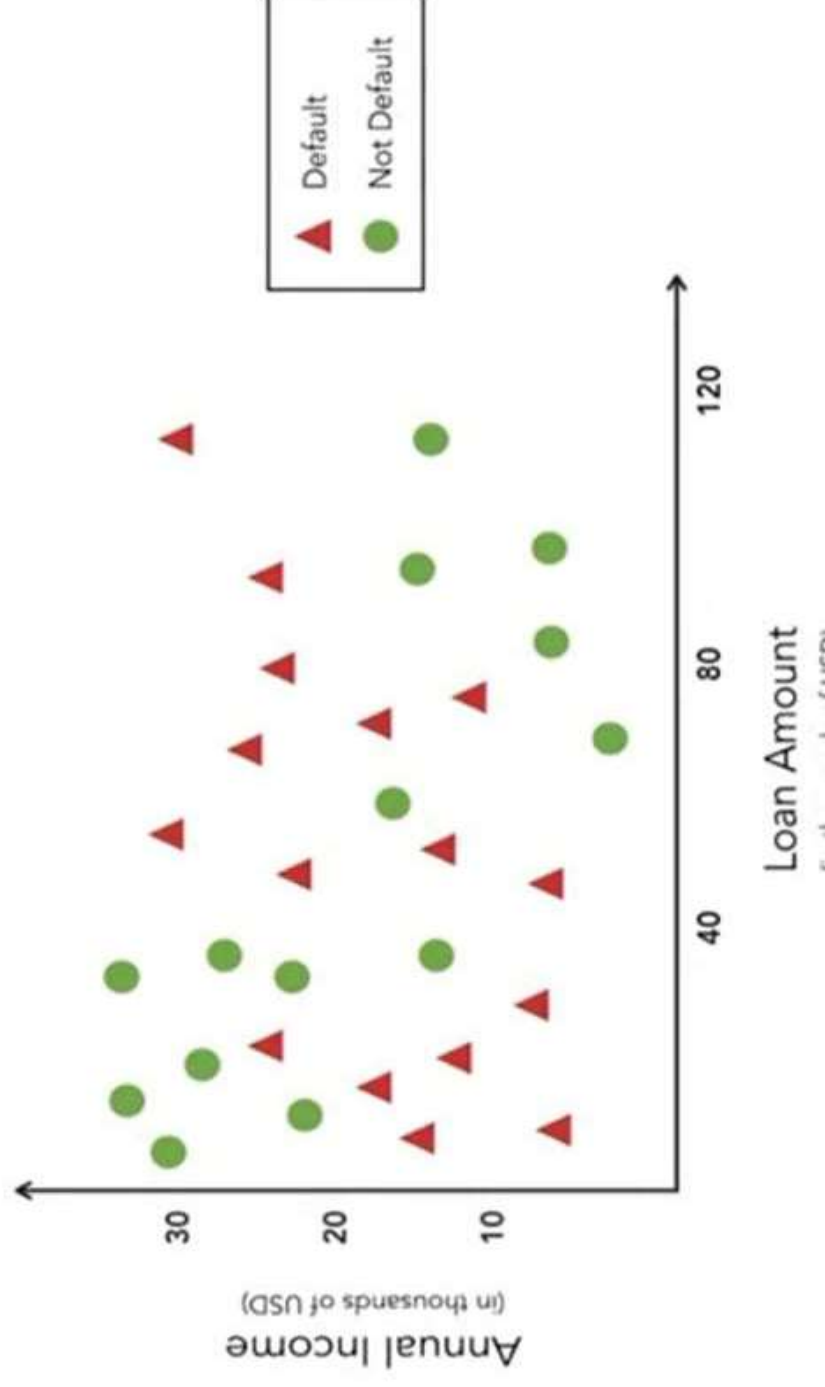


Customer	Income	Loan Amount	Degree
James S.	\$30000	\$8000	No
Carol J.	\$28000	\$20000	No
Sarah P.	\$18000	\$15000	Yes
John J.	\$12000	\$21000	Yes
Saul B.	\$13000	\$35000	No
Alan Q.	\$13000	\$45000	Yes
Frank D.	\$15000	\$60000	No
Jane R.	\$26000	\$70000	Yes
Bill T.	\$28000	\$110000	Yes
Tom C.	\$16000	\$100000	No

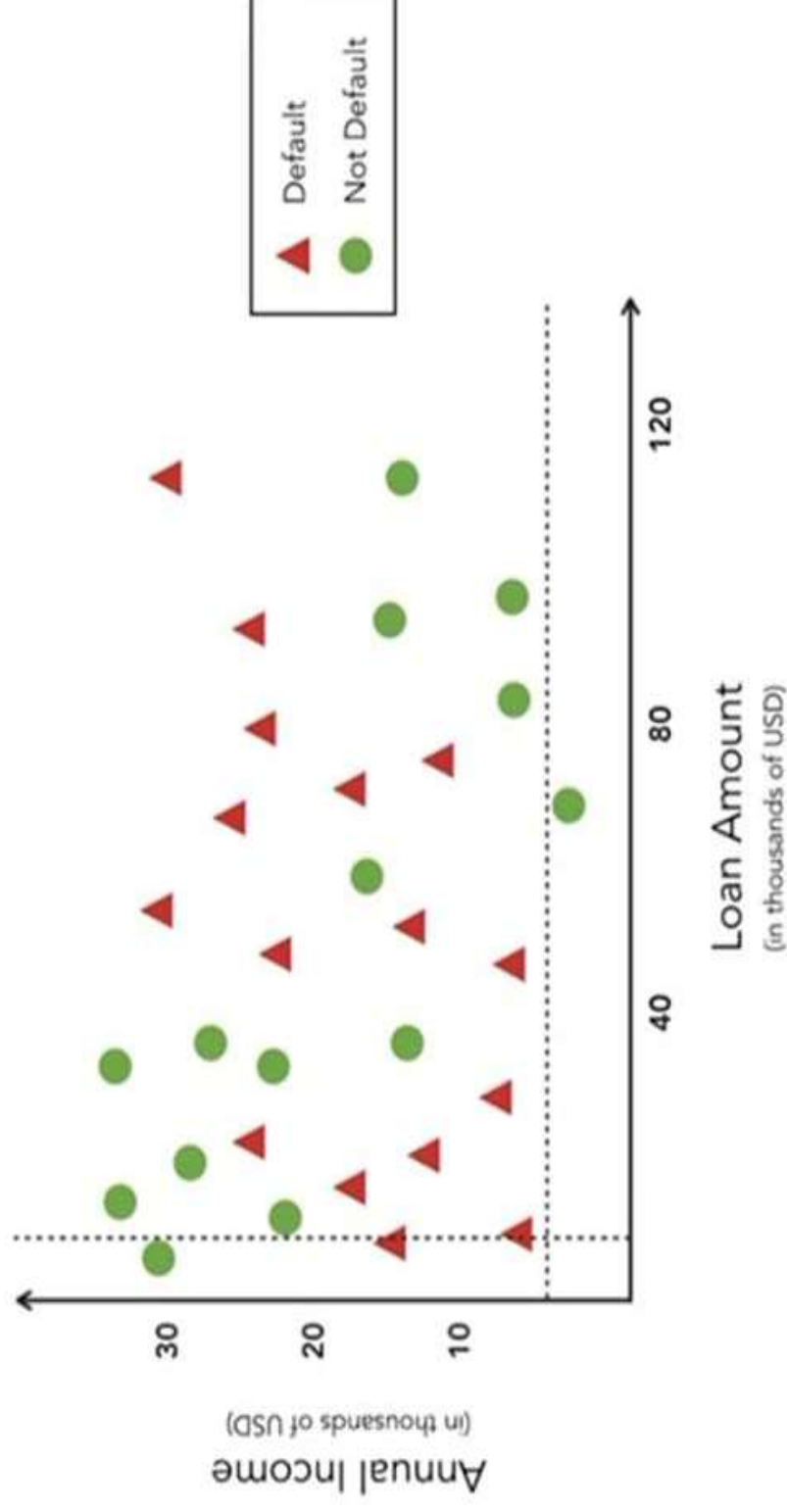


Dependent Variable

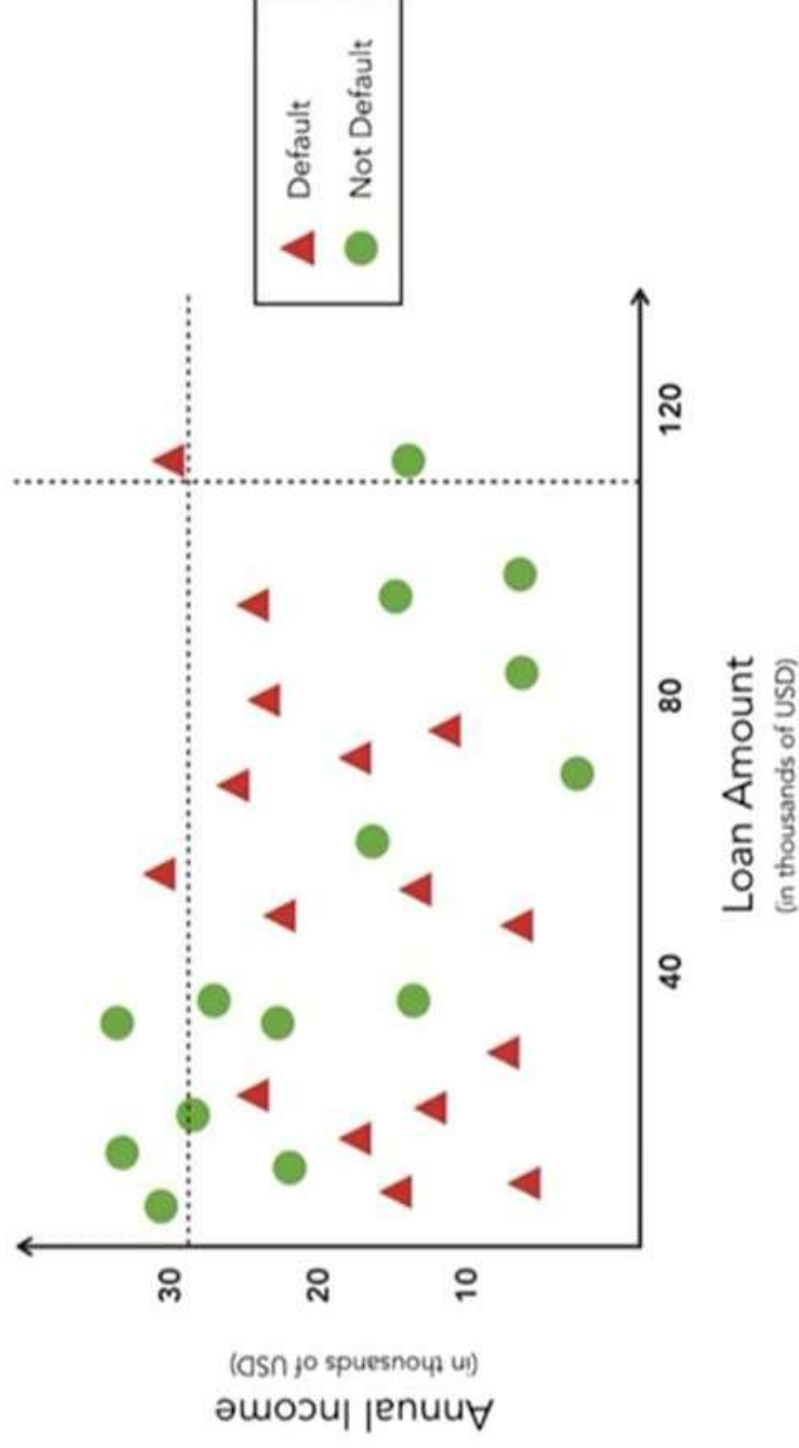
Recursive Partitioning



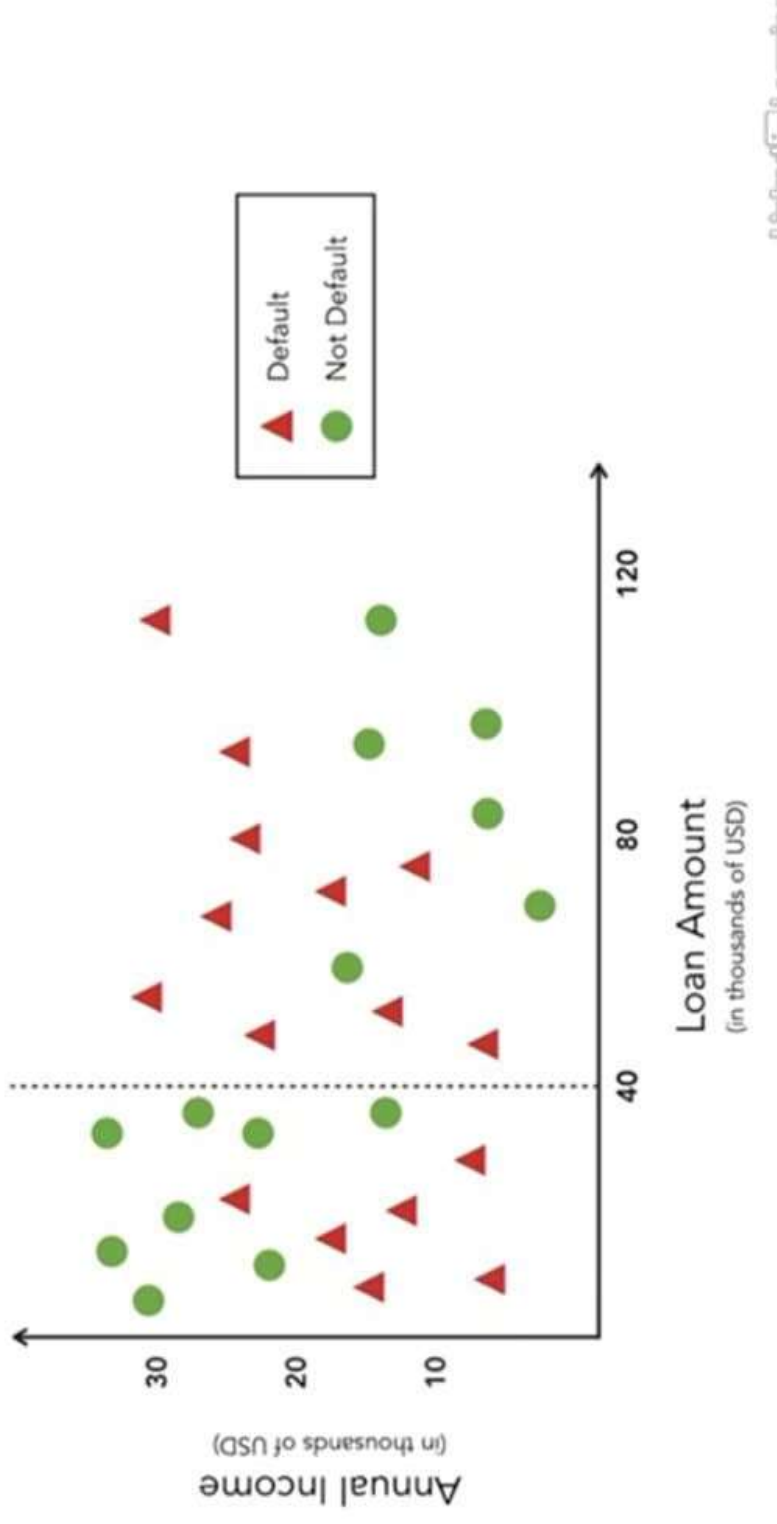
Recursive Partitioning



Recursive Partitioning



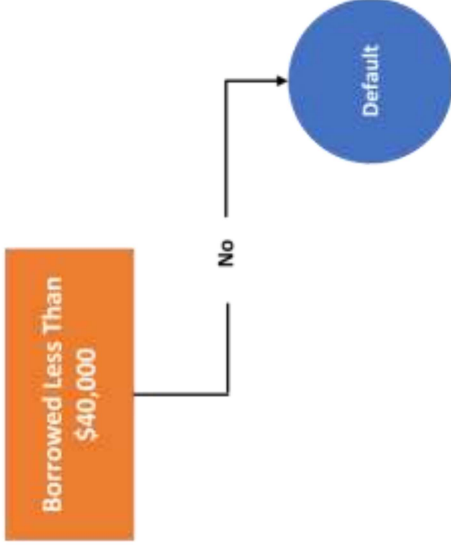
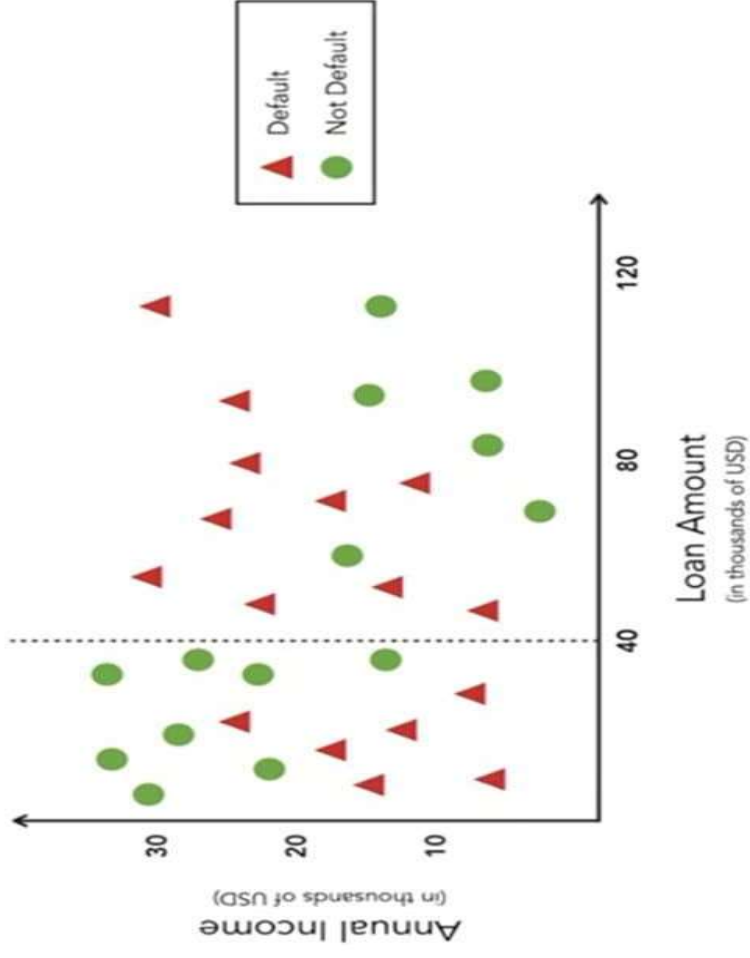
Recursive Partitioning



Splitting the Data (Recursive Partitioning)

- **Step 1:** We look for the best way to **split the data** to create groups that are mostly similar (called "purity").
- Example: Split based on whether the loan amount is **\$40,000 or less**.
- This creates two groups: Loans $\leq$ \$40,000 and Loans $>$ \$40,000.

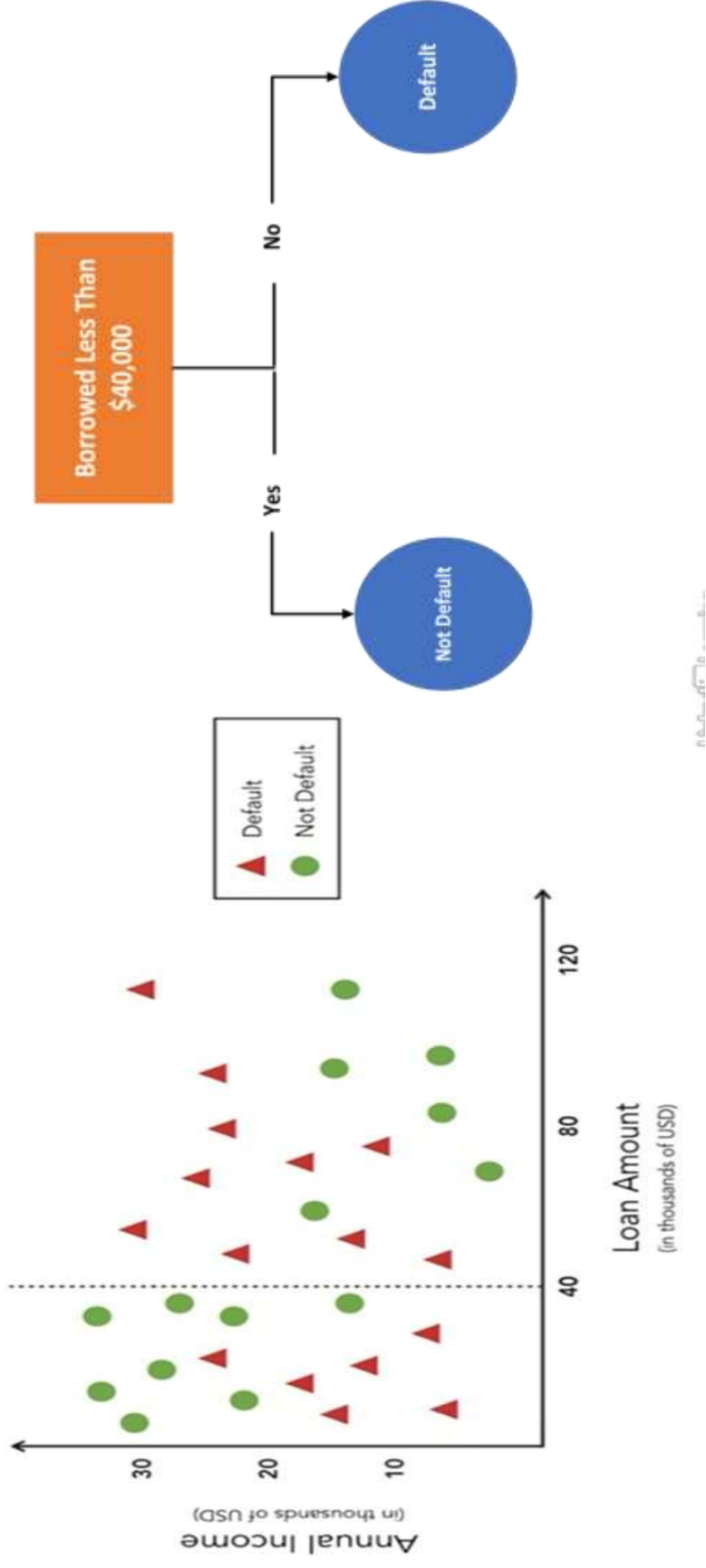
Recursive Partitioning



Creating the First Decision Node

- The first **decision node** asks, "Did the customer borrow **\$40,000 or less?**"
- If **Yes**: We assume most loans will be **paid back**.
- If **No**: We assume most loans will **default**.
- These assumptions create the **branches** and **leaf nodes** of the tree.

Recursive Partitioning

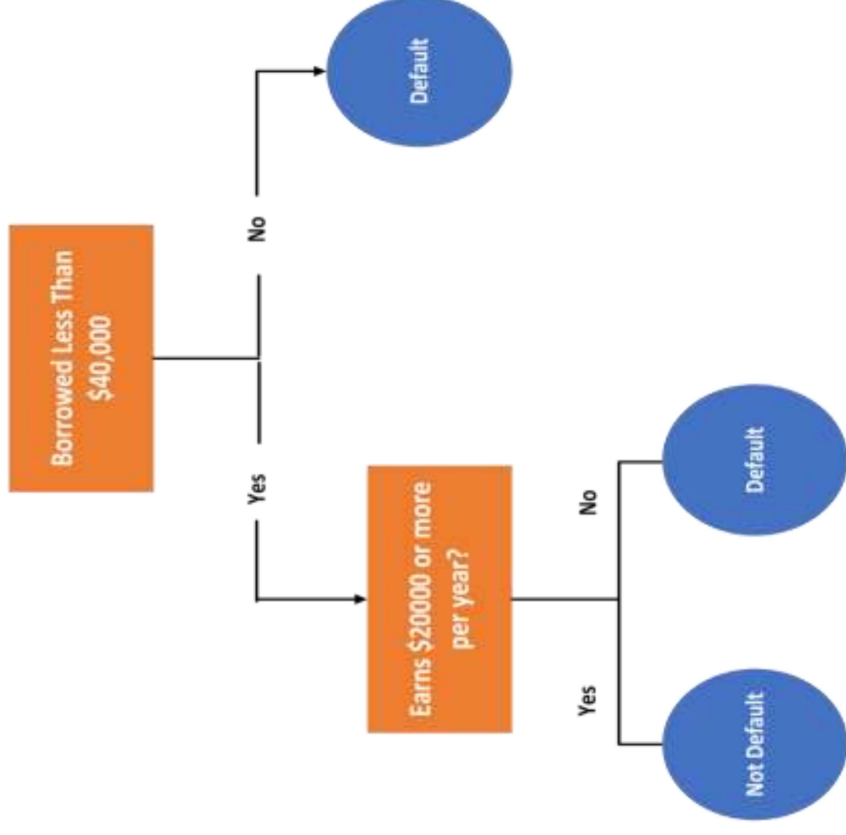
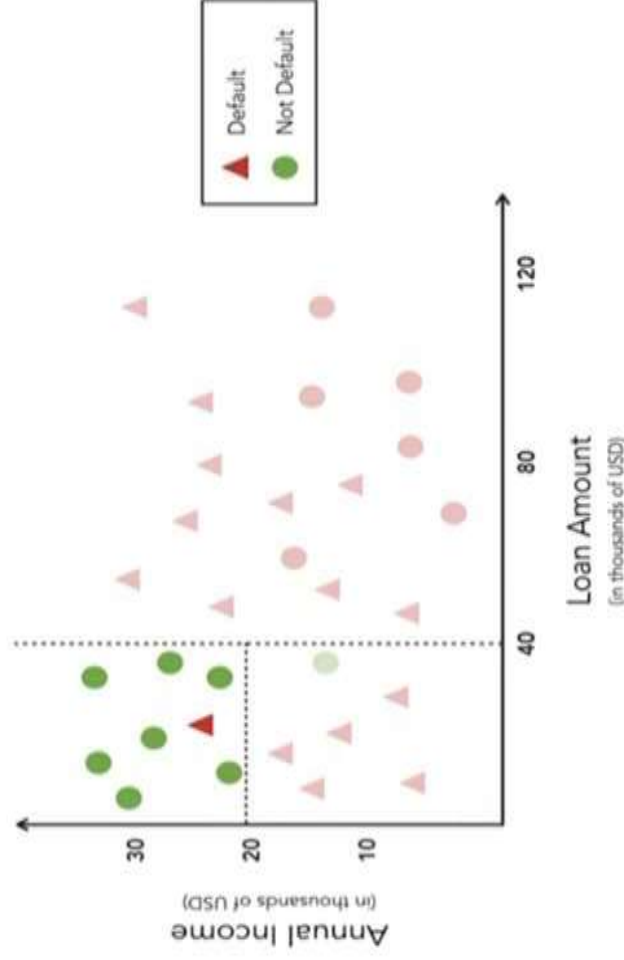


Further Splitting the Data

- If the data still has a mix of results (some paid back, some defaulted), we can **further split** the data.
- Example: Split based on **income** (e.g., over or under **\$20,000**).
- This reduces the number of **misclassified examples** and makes the groups more accurate.

Further Splitting the Data

Recursive Partitioning



When to Stop Building the Tree?

- Recursive partitioning continues until: **All items** in a group are the same (e.g., all defaulted).
- There are **no more variables** left to split on.
- A **condition** you set (like tree depth) is met.

Measuring Impurity in Classification Tree

- **Impurity** measures how mixed the data is in each partition.
- Classification trees aim to create **purier partitions** as they split.
- Two common impurity measures: **Entropy** (used in C5.0 algorithm)
- **Gini** (used in CART algorithm)

Entropy Explained

- **Entropy** quantifies the **randomness or disorder** in a partition.
- **Low entropy**: Items are similar (purer partition).
- **High entropy**: Items are more mixed (less pure).

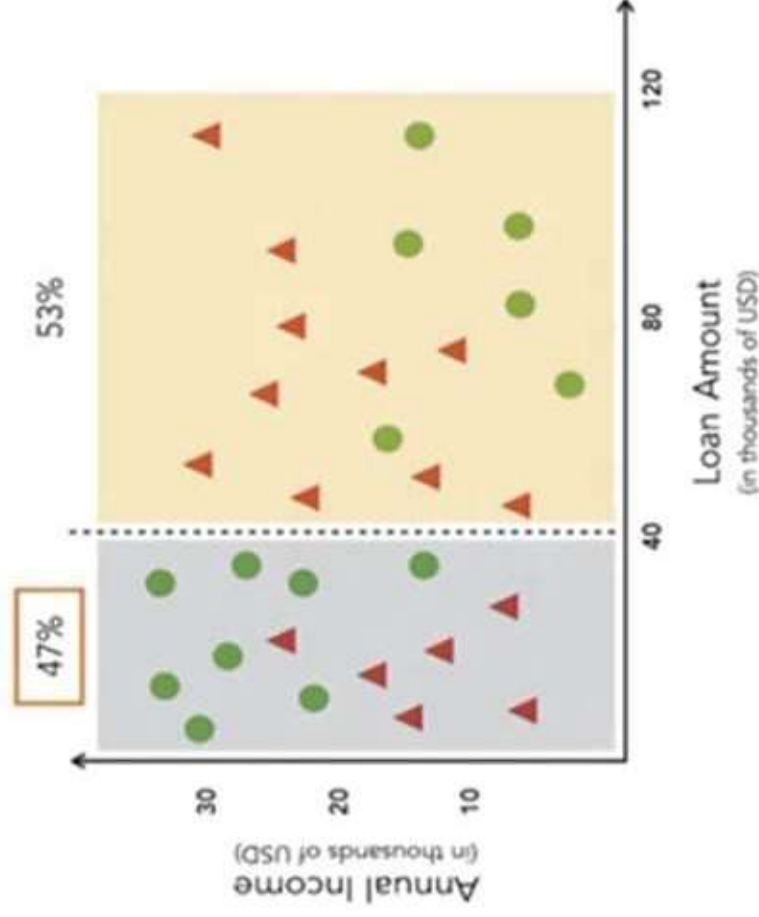
Calculating the Entropy

- Formula: $\text{Entropy}(S) = -\sum p_i \log_2 p_i$
- Example: Two outcomes, Red Triangles and Green Circles.
 - Proportion of Red Triangles: $16/30 = 0.5316/30 = 0.5316/30 = 0.53$
 - Proportion of Green Circles: $14/30 = 0.4714/30 = 0.4714/30 = 0.47$
- **Entropy = 0.997**

How Entropy Helps Splitting?

- When splitting, the tree calculates the **entropy of each partition**.
- Example: Splitting on a loan amount of **\$40,000**: Entropy of left partition: **0.986**
- Entropy of right partition: **0.951**
- The weighted average gives a **combined entropy** after the split.

Entropy as a Measure of Impurity



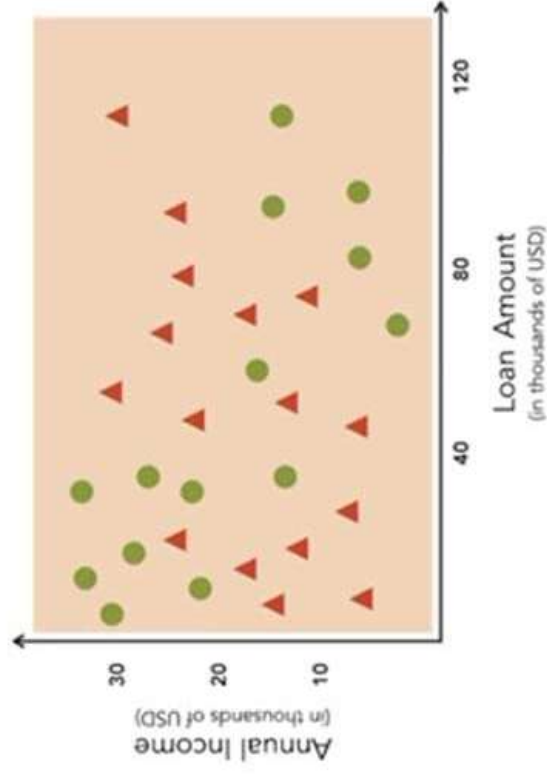
$$\begin{aligned}
 \text{Entropy}(S_2) &= \sum_{i=1}^n w_i \times \text{Entropy}(P_i) \\
 &= 0.47 \times 0.986 + 0.53 \times 0.951 \\
 &= 0.967
 \end{aligned}$$

Information Gain

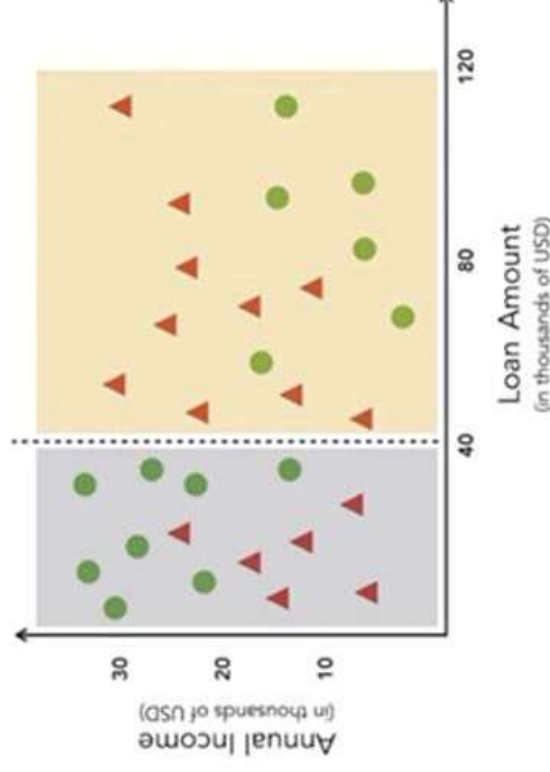
- **Information gain** is the reduction in entropy after a split. Example: Before the split, entropy is **0.997**.
- After the split, combined entropy is **0.967**.
- **Information gain** = **0.997 - 0.967 = 0.03**
- The algorithm chooses the split with the **largest information gain**.

Information Gain

Entropy as a Measure of Impurity



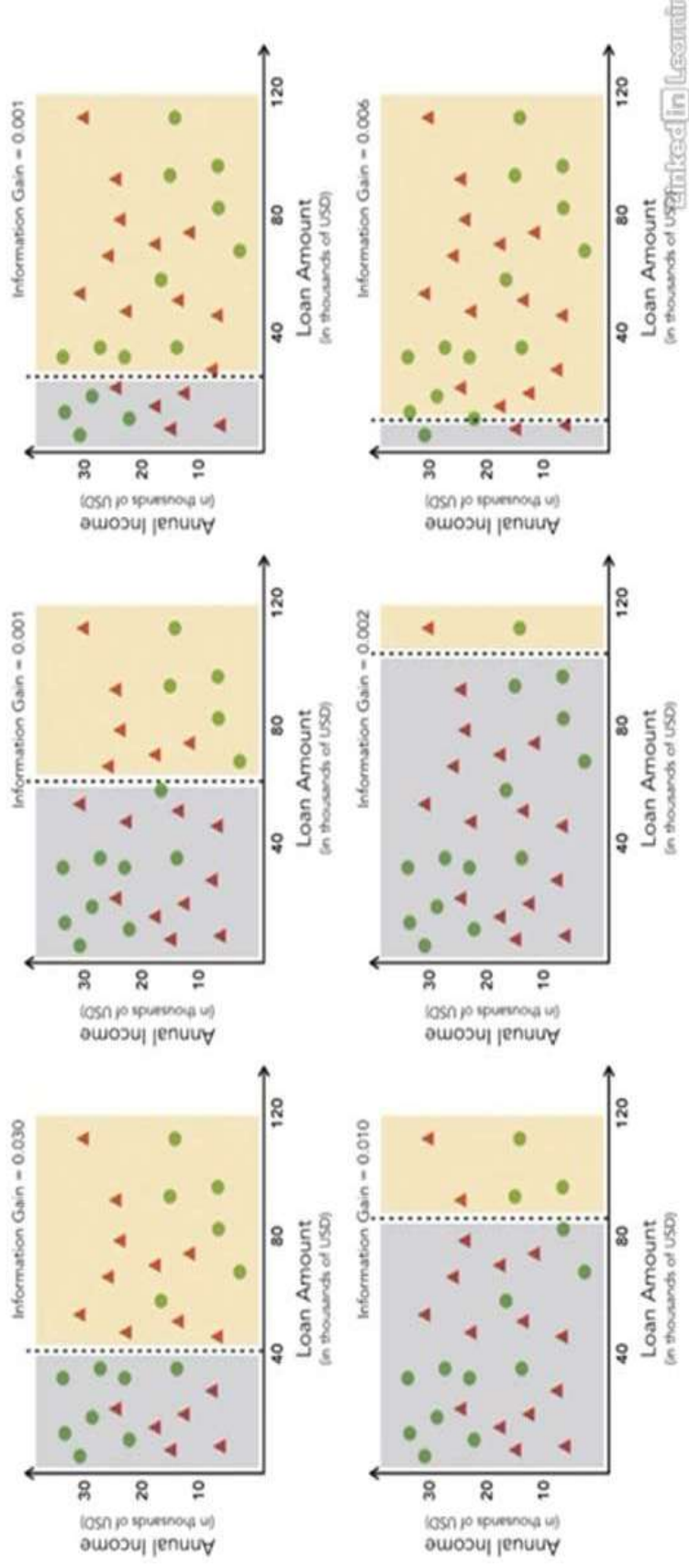
$$\text{Entropy}(S_1) = 0.997$$



$$\text{Entropy}(S_2) = 0.967$$

Information Gain

Information Gain



Gini Index

- **Gini** measures the likelihood of incorrect labeling if done randomly.
- **High Gini**: More random (impure partition).
- **Low Gini**: Less random (purer partition).
- **Gini** = $1 - \sum p_i^2$ (sum of squared proportions).

Using Gini to Split

- Just like entropy, Gini helps decide where to split.
- The algorithm calculates **Gini values** for potential splits and chooses the one with the **largest reduction**.